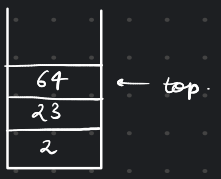
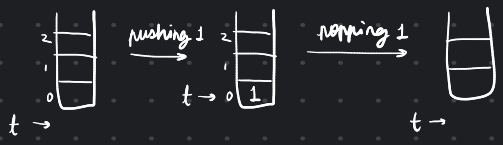
॥ श्रीगणेशाय नमः ॥

Stacks and Queues

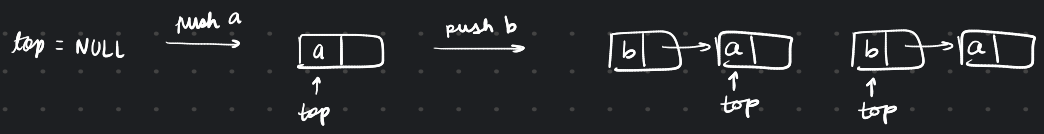
Stacks

These are last-in-first-out data structures. They use a 'top' variable to indicate the top of the stack.
Initially, the top is at index -1.





Stacks and queues can be represented using arrays or linked lists.
In arrays, the top will be the last index. For a linked-list stack, initially top = NULL, later it would point to the head.



Array implementation

```
PUSH(key):
    if top == n-1:
        return "Stack is full"
    else:
        top += 1
        stack[top] = key
```

```
POP():
    if top == -1:
        return "Stack is empty"
    else:
        temp = stack[top]
        top -= 1
        return temp
```

Linked list implementation

```
def push(top, data):
    if top is None:
        top = Node(data)
        return top

    new_node = Node(data)
    new_node.next = top
    top = new_node
    return top
```

```
def pop(top):
    if top is None:
        return "stack underflow"
    temp = top
    top = top.next
    temp.next = None
    return top
```

Queues

These are first-in-first-out data structures. They use 'front' and 'rear' variables.
Insertion (enqueue) occurs at the front, deletion (dequeue) occurs at the rear.




```
ENQUEUE(key):
    if rear+1 == N:
        return "overflow"

    if rear == -1:
        front += 1

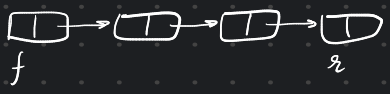
    rear += 1
    queue[rear] = key
```

```
DEQUEUE():
    if front == -1:
        return "underflow"

    key = queue[front]

    if front == rear:
        front = rear = -1
    else:
        front += 1
```

Linked list implementation




```python
def enqueue(front, rear, key):
    new_node = Node(key)

    if front is None:
        front = rear = new_node
    else:
        rear.next = new_node
        rear = new_node
```

```python
def dequeue(front, rear):
    if front is None:
        return None

    key = front.data
    front = front.next
# in Python, we don't care about freeing memory
# as the garbage collector will take care of it
```

The time complexity of all of these operations is O(1).

8.12.5 Stack: GATE CSE 2003 | Question: 64

Let **S** be a stack of size $n \geq 1$. Starting with the empty stack, suppose we push the first n natural numbers in sequence, and then perform n pop operations. Assume that Push and Pop operations take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For $m \geq 1$, define the stack-life of m as the time elapsed from the end of $Push(m)$ to the start of the pop operation that removes m from **S**. The average stack-life of an element of this stack is

A. $n(X+Y)$ B. $3Y+2X$ C. $n(X+Y)-X$ D. $Y+2X$

gatecse-2003 data-structures stack normal

push push push pop pop pop
x y x y x y x y x y x y

In-fix to post-fix conversion

a + b * c ⟶ a b c * +

ALGORITHM
a. Create a stack.
b. For each character t in input:
    if t is an operand:
        output t
    else if t is a right paranthesis:
        pop and output tokens until a left paranthesis is popped (but not output)
    else if t is operator or left paranthesis:
        pop and output tokens until one of the lower priority than t is encountered
            or a left paranthesis is encountered or stack is empty
        push t

Operators	Associativity
() [] -> .	left to right
! ~ ++ -- + - * (type) sizeof	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
&	left to right
^	left to right
|	left to right
&&	left to right
||	left to right
?:	right to left
= += -= *= /= %= &= ^= |= <<= >>=	right to left
,	left to right



- a higher priority operator should sit on top of any lower priority operator.

(to push a lower priority operator, pop the higher priority ones below first)

< R to L > L to R

(Right operator has greater priority when both are at the same level

Q. a * b + c

↑ ↑ ↑ ↑ ↑ pop everything
push pop

a b * c +



Q. d * a - b / c + e = f = g * h

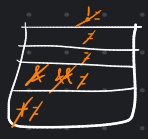
↑↑ ↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑

d a * b c / - e + f g h * = =



Q. a + b = c & d && e = f = g = h != j

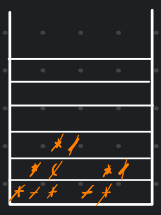
↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑



a b + c d & e && f g h j != = = = =

Q. 2 + 3 - 1 * 2 + (8 * 2 / 16) - 3 + 4 * 3 / 6

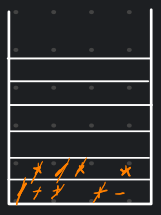
↑ ↑



2 3 + 1 2 * - 8 2 * 16 / + 3 - 4 3 * 6 / +

Q. 10 / 5 - 3 * 7 + 8 / 1 * 9 + 3 - 4 * 7

↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑



10 5 / 3 7 * - 8 1 / 9 * + 3 + 4 7 * -

Evaluation to Postfix



1 2 3 * +

↑ push ↑ push ↑ push ↑ pop 2 ↑ pop 2

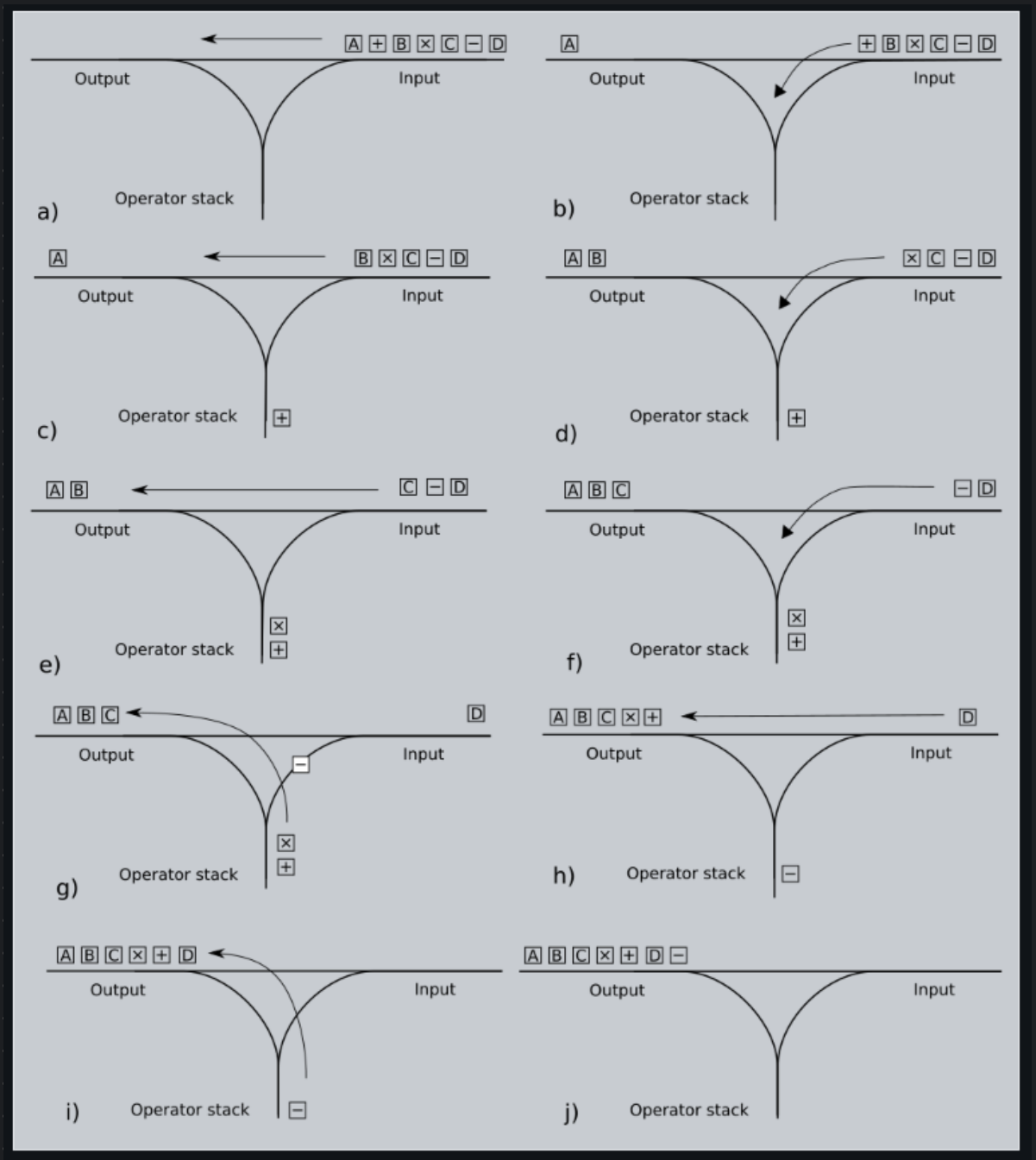
(6,1) (6+1) ← push

(3,2) & push result

(3 * 2) ← push

whatever's left in the end is the result = 7

ALGORITHM
1. Scan the postfix from left to right.
2. Initialize an empty stack.
3. Repeat steps 4 and 5 till all the characters are scanned.
4. If the scanned character is an operand, push it onto the stack.
5. If the scanned character is an operator:
    If the operator is unary, pop an element off the stack.
    If the operator is binary, pop 2 elements off the stack.
    After popping the element(s), apply the operator to the popped element(s).
    Push the result onto the stack.


A + B × C − D
Output
Input
Operator stack
a)
A
+ B × C − D
Output
Input
Operator stack
b)
A
B × C − D
Output
Input
Operator stack
+
c)
A B
× C − D
Output
Input
Operator stack
+
d)
A B
C − D
Output
Input
Operator stack
×
+
e)
A B C
− D
Output
Input
Operator stack
×
+
f)
A B C
D
Output
Input
−
Operator stack
×
+
g)
A B C × +
D
Output
Input
Operator stack
−
h)
A B C × + D
Output
Input
Operator stack
−
i)
A B C × + D −
Output
Input
Operator stack
j)

Input: 3 + 4 × 2 ÷ (1 − 5) ^ 2 ^ 3

Token	Action	Output (in RPN)	Operator stack	Notes
3	Add token to output	3		
+	Push token to stack	3	+	
4	Add token to output	3 4	+	
×	Push token to stack	3 4	× +	× has higher precedence than +
2	Add token to output	3 4 2	× +	
÷	Pop stack to output	3 4 2 ×	+	÷ and × have same precedence
	Push token to stack	3 4 2 ×	÷ +	÷ has higher precedence than +
(	Push token to stack	3 4 2 ×	(÷ +	
1	Add token to output	3 4 2 × 1	(÷ +	
−	Push token to stack	3 4 2 × 1	− (÷ +	
5	Add token to output	3 4 2 × 1 5	− (÷ +	
)	Pop stack to output	3 4 2 × 1 5 −	(÷ +	Repeated until "(" found
	Pop stack	3 4 2 × 1 5 −	÷ +	Discard matching parenthesis
^	Push token to stack	3 4 2 × 1 5 −	^ ÷ +	^ has higher precedence than ÷
2	Add token to output	3 4 2 × 1 5 − 2	^ ÷ +	
^	Push token to stack	3 4 2 × 1 5 − 2	^ ^ ÷ +	^ is evaluated right-to-left
3	Add token to output	3 4 2 × 1 5 − 2 3	^ ^ ÷ +	
end	Pop entire stack to output	3 4 2 × 1 5 − 2 3 ^ ^ ÷ +		

Implementing a queue using 2 stacks



enqueue a, b, c



dequeue



enqueue d, e, f



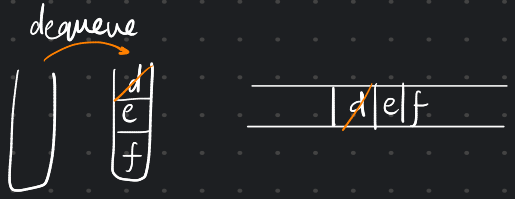
dequeue



dequeue



dequeue




```
stack1 = []
stack2 = []

def enqueue(element):
    stack1.append(element)

def dequeue():
    if len(stack2) == 0:
        if len(stack1) == 0:
            return None
        while len(stack1) > 0:
            element = stack1.pop()
            stack2.append(element)
    if len(stack2) == 0:
        return None
    return stack2.pop()
```

Q. A queue is implemented using 2 stacks. The following operations are done. What are the # of pushes ad pops performed?
15 enqueues, 4 dequeues, 25 enqueues, 19 dequeues.

→

15 enqueues → 15 pushes [15] [0]

4 [1 dequeue → 15 pops, 15 pushes, 1 pop [0] [14]
3 dequeues → 3 pops [0] [11]

25 enqueues → 25 pushes [25] [11]

19 [11 dequeues → 11 pops [25] [0]
1 dequeue → 25 pops, 25 pushes, 1 pop [0] [24]
7 dequeues → 7 pops [0] [17]

80 pushes
63 pops